

The background of the slide is a close-up, slightly blurred photograph of a dictionary page. The word 'monitor' is printed in a bold, black font and is highlighted with a thick black dot to its right. Above it, the word 'warn' is visible in quotes, and the word 'ORIGIN' is partially visible at the top. Below 'monitor', the text 'thing 2 a software' and 'duties 3 a software' are partially legible. The overall color scheme is a cool blue-grey.

BCC2001

Diego Bertolini
diegobertolini@gmail.com

Aula

■ **Estrutura de Dados Composta**

- Vetores em algoritmos.
- Vetores em C.

Estrutura de dados composta

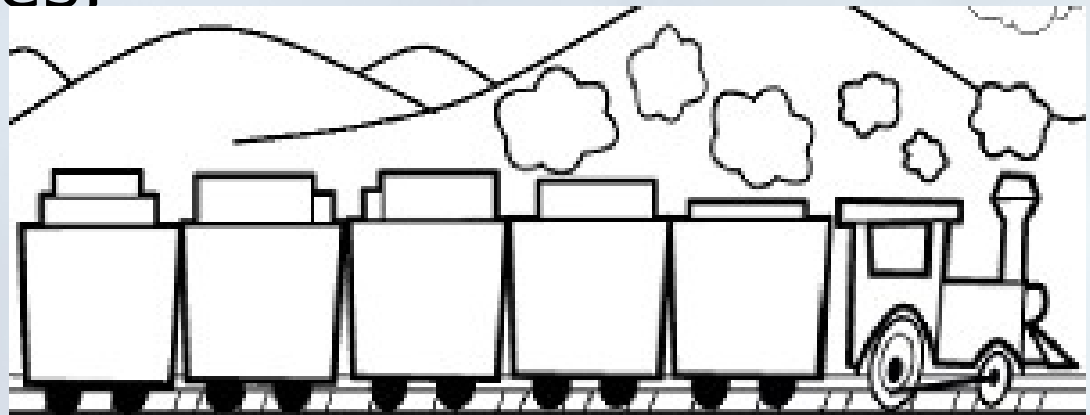
- São um conjunto de variáveis identificadas por um mesmo nome.
- Homogêneas
 - Arranjos unidimensionais → vetores
 - Arranjos bidimensionais → matrizes
 - Multidimensionais
- Heterogêneas
 - Estruturas → structs → registros

Agregados Homogêneos

- Até agora os problemas eram resolvidos com tipos de dados simples (ou primitivos);
- Agora estudaremos os tipos de dados estruturados;
- **Tipos de Dados Estruturados:**
 - Agregados homogêneos (sequência de valores de um mesmo tipo);
 - Agregados heterogêneos (sequência de valores de diferentes tipos);

Agregado Homogêneo

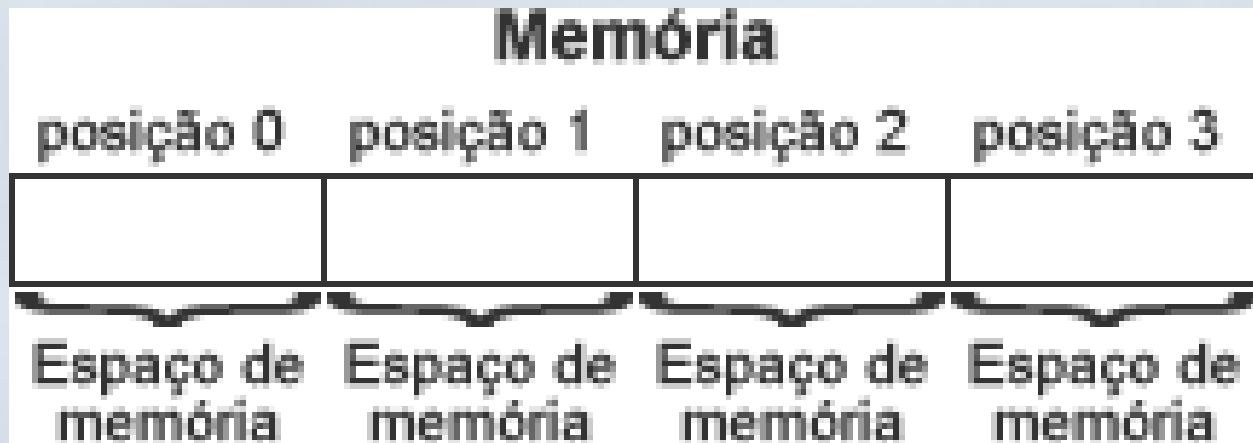
- **Agregado homogêneo:** é uma sequência de dados do mesmo tipo que podem ser associada à um único identificador;
- Para utilizarmos um agregado homogêneo, devemos declará-lo estabelecendo o tipo de seus componentes, e o seu número máximo de componentes:



Agregado Homogêneo

■ Exemplo de manipulação:

Posição	0	1	2	3	4	5	6	7	8	9
Idades->	55	32	50	10	79	7	19	29	79	87



Agregado Homogêneo

■ Exemplo de manipulação:

vetor[2] = 50;

Índice	Array									
	0	1	2	3	4	5	6	7	8	9
			50							

Como declarar um vetor

- Utilizados para armazenar conjuntos de dados cujos elementos podem ser endereçados por uma única dimensão, um único índice.
- Também conhecidos como *vetores* ou *array*.

Como declarar:

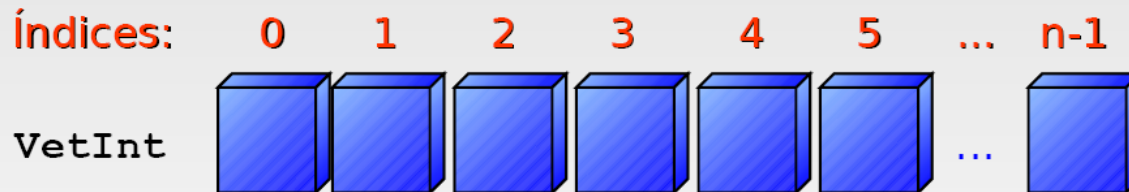
<tipo> nome [<tamanho>]

Exemplos:

```
float vetorFloat[50];  
int idades[44];  
char nomeAluno[100];
```


Como declarar um vetor

```
int VetInt[n];
```



Índice do **primeiro** elemento: **zero**

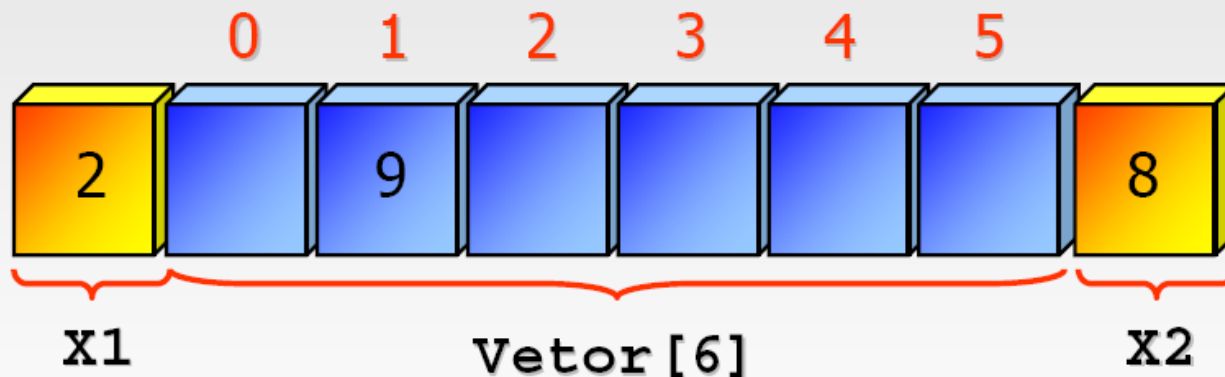
Índice do **último** elemento: **n - 1**

Quantidade de elementos: **n**

- O espaço ocupado na memória, em bytes é:
 - $\text{espaço} = \text{tamanho} * \text{sizeof}(\text{tipo})$

Atenção aos limites

- Como o acesso é feito a posições de memória indexadas pelo índice, atenção para os limites das estruturas declaradas.



```
int X1;  
int Vetor[6];  
int X2
```

```
Vetor[1] = 9;  
Vetor[-1] = 2;  
Vetor[6] = 8;
```

Características importantes!

- Cada posição do array possui todas as características de uma variável ; Isso significa que ela pode ser usada em comandos de entrada e saída, expressões e atribuições ;
 - `scanf("%d", ¬as[5]) ;`
 - `notas[0] = 10 ;`
 - `notas[1] = notas[5] + notas[0]`

Atividade - Exemplos

1. Faça um algoritmo que armazene as temperaturas diárias colhidas em um período de 7 dias. Ao final, o algoritmo deverá informar em quantos dias a temperatura foi acima de 30º.
1. Faça um programa em C que permita ao usuário cadastrar o preço de compra e o preço de venda de 10 produtos. Ao final, o algoritmo deverá informar o valor do lucro obtido com a venda de uma unidade de cada um dos 10 produtos.

Atividade 01

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     float vetorTemp[20];
7     int i, contDias = 0 ;
8
9     for (i = 0; i<20; i++)
10    {
11        printf("Informe a Temperatura:");
12        scanf("%f", &vetorTemp[i]);
13
14        if (vetorTemp[i] < 0)
15            contDias++;
16    }
17
18    printf("Numero de Dias com Temp Negativas %d\n\n", contDias);
19    system("PAUSE");
20
21 }
```

Atividade 02

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     float vetorPrecoCusto[10];
7     float vetorPrecoVenda[10];
8     int i, contDias = 0 ;
9     float somaCusto = 0;
10    float somaVenda = 0;
11
12    for (i = 0; i<10; i++)
13    {
14        printf("Informe o Preco de Custo: ");
15        scanf("%f", &vetorPrecoCusto[i]);
16        printf("Informe o Preco de Venda: ");
17        scanf("%f", &vetorPrecoVenda[i]);
18        somaCusto = somaCusto + vetorPrecoCusto[i];
19        somaVenda = somaVenda + vetorPrecoVenda[i];
20    }
21
22    printf("Valor do Lucro (Venda - Custo) %f\n\n", somaVenda - somaCusto);
23    system("PAUSE");
24
25 }
```

Vetores

- O tamanho de um vetor é pre-definido, ou seja, após a compilação, não pode ser alterado.
- A declaração e a alocação são estáticas.
 - São estruturas de dados estáticas: mantém o mesmo tamanho ao longo de toda a execução do programa.

Formas de Inicialização

- Atribuir valores na própria declaração do vetor:
- `int erro[5];`
- `erro = { 2, 4, 6, 8, 10 }; /* ISTO ESTÁ ERRADO */`
- `int x = 21;`
- `int yy[3] = { 1, 2, x }; /* ISTO ESTÁ ERRADO */`

`int` vetor[5] = {1, 2, 3, 4, 5};

Formas de Inicialização

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int vetorPrecoCusto[10], i ;
7
8     for (i = 0; i<10; i++)
9     {
10         vetorPrecoCusto[i] = 0;
11         printf("Indice: %d - Conteudo:%d \n", i, vetorPrecoCusto[i]);
12     }
13
14     system("PAUSE");
15
16 }
```

Carregando Valores

- O usuário atribui valores:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int vetorPrecoCusto[10], i ;
7
8     for (i = 0; i<10; i++)
9     {
10         printf("Digite o Valor: ");
11         scanf("%d", &vetorPrecoCusto[i]);
12         printf("Indice: %d - Conteudo:%d \n", i, vetorPrecoCusto[i]);
13     }
14
15     system("PAUSE");
16
17 }
```

Percorrer e imprimir ...

■ Imprimindo os elementos

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int vetor[5] = {1, 2, 3, 4, 5}, i ;
7
8     for (i = 0; i<5; i++)
9     {
10         printf("Indice: %d - Conteudo:%d \n", i, vetor[i]);
11     }
12
13     system("PAUSE");
14
15 }
```

Agregado homogêneo - String

- **Cadeia de Caracteres**

- É um outro tipo de estrutura de dados homogênea;
- Também conhecidas como **strings**, são estruturas homogêneas que permitem especificamente o armazenamento de sequências de caracteres;
- O tamanho máximo da sequência de caracteres a ser armazenada deve ser definido na declaração da cadeia;
- **Exemplo:**
 - `char nome[12];`
 - Se armazenássemos a palavra “computador” nesta estrutura teríamos:

C	O	M	P	U	T	A	D	O	R		
---	---	---	---	---	---	---	---	---	---	--	--



Neste caso diz-se que o tamanho físico da estrutura é 12 e o tamanho lógico é 10.

Strings

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     char nome[12] = {"Computador"}, i ;
7
8     for (i = 0; i<12; i++)
9     {
10         printf("Indice: %d - Conteudo:%c \n", i, nome[i]);
11     }
12
13     system("PAUSE");
14
15 }
```

```
Indice: 0 - Conteudo:C
Indice: 1 - Conteudo:o
Indice: 2 - Conteudo:m
Indice: 3 - Conteudo:p
Indice: 4 - Conteudo:u
Indice: 5 - Conteudo:t
Indice: 6 - Conteudo:a
Indice: 7 - Conteudo:d
Indice: 8 - Conteudo:o
Indice: 9 - Conteudo:r
Indice: 10 - Conteudo:
Indice: 11 - Conteudo:
Pressione qualquer tecla
```

Atividade

- Faça um algoritmo em C que pergunte o nome do usuário e em seguida escreva na tela Ola “nome do usuário”.

Atividade

- Faça um algoritmo para armazenar uma palavra com no máximo 15 caracteres a ser digitada pelo usuário, e informar a quantidade de caracteres 'a' que aparece na palavra:

Atividades

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     char nome[12] ;
7
8     printf("Digite o nome do usuario: ");
9     scanf("%s",&nome);
10    printf("Ola %s ", nome);
11    printf("\n\n");
12
13    system("PAUSE");
14
15 }
16 |
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     char palavra[14] ;
7     int i, contLetra = 0 ;
8     printf("Digite a Palavra: ");
9     scanf("%s", &palavra);
10
11    for (i = 0; i < 15; i++)
12    {
13        if (palavra[i] == 'a')
14            contLetra = contLetra + 1;
15    }
16
17    printf("Quantidade de Letras a: %d ", contLetra);
18    printf("\n\n");
19
20    system("PAUSE");
21
22 }
```

Atividade

- Faça um algoritmo que permite ao usuário digitar 10 valores inteiros e depois exibir a sequência em ordem invertida.
- **Entrada:**
 - {1,2,3,4,5,6,7,8,9,10}
- **Saída:**
 - {10,9,8,7,6,5,4,3,2,1}

Atividade

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int num[10] = {1,2,3,4,5,6,7,8,9,10}, i ;
7
8     for (i = 9; i >= 0; i--)
9     {
10         printf("Indice: %d - Valor: %d\n",i,num[i]);
11     }
12
13     printf("\n\n");
14
15     system("PAUSE");
16
17 }
```

Características importantes!

- Sempre que estiver declarando um número grande de variáveis do mesmo tipo e para um mesmo fim, provavelmente você poderá utilizar um array.
- Declaração: `tipo_dado NomeArray[tamanho] ;`
- O tamanho sempre deve ser um valor ou expressão inteira e constante ;
- Para indicar qual índice queremos acessar, utiliza-se o operador colchetes `[ ]`. Ex. `nota[índice] ;`
- O uso de array permite usar comandos de repetição sobre as tarefas que devem ser realizadas, de forma idêntica, para cada posição do array, apenas modificando o índice do array.

Características importantes!

- Cada posição do array é uma variável, a qual é identificada por um índice. O tempo para acessar qualquer uma das posições do array é o mesmo.
- Na linguagem C, a numeração do índice do array começa sempre do ZERO e termina sempre em $N-1$, em que N é o número de elementos definido na declaração do array ;
- Em um array de 100 elementos, índices menores do que 0 e maiores que 99 também podem ser acessados. Porém isso pode resultar nos mais variados erros durante a execução do programa.

Dúvidas, Críticas ou Sugestões

diegobertolini@utfpr.edu.br